

使用 Messaging 和 People API，在 Android 應用程式中新增即時通訊相關功能

來源：<https://codelabs.developers.google.com/people-messaging?hl=zh-tw>

步驟數：7

瞭解如何擴充 Android 應用程式，透過 Messaging 和 People API 加入即時通訊相關功能。

目錄

1. [1. 事前準備](#)
2. [2. 做好準備](#)
3. [3. 建立對話通知](#)
4. [4. 為對話建立分享目標](#)
5. [5. 選用：啟用泡泡](#)
6. [6. 選用：分享連結](#)
7. [7. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 事前準備

建立訊息應用程式並不容易。雖然整體使用者體驗是主觀的，但 Android 提供 Messaging 和 People API，可改善及簡化即時通訊體驗。

在本程式碼研究室中，您將瞭解如何使用這些 API，在 Android 上為即時通訊應用程式建立引人入勝的生態系統。您將擴充 [JetChat](#) 應用程式，這是一款使用 Jetpack Compose 的基本即時通訊應用程式，不具備任何功能。

必要條件

- 具備 Android 開發的基本知識
- 具備[通知](#)基本知識

建構項目

擴充的 JetChat 應用程式，可執行下列操作：

- 在通知導覽匣的保留對話部分，顯示代表對話的通知。
- 這些通知會分享目標參照，方便您將內容分享到應用程式的對話中。
- 強制執行這些物件的建立最佳做法，充分運用系統提供的預設體驗，提升應用程式品質。

課程內容

- 如何讓通知導覽匣的保留對話部分顯示對話相關通知。
- 如何瞭解 Messaging 和 People API 支援的各種體驗。

軟硬體需求

- Git
 - Android Studio
 - GitHub 帳戶
-

步驟 2 · 約 4 分鐘

2. 做好準備

起點是 [JetChat 應用程式](#)。範例程式碼會擴充 JetChat 應用程式，以便更清楚展示 Messaging 和 People API。

取得範例程式碼

如要取得本程式碼研究室的範例程式碼，請按照下列步驟操作：

1. 在指令列中複製下列 GitHub 存放區：



```
git clone --branch starter-code \  
https://github.com/android/people-messaging-codelab.git
```

2. 在 Android Studio 中開啟專案，然後按一下「Run app」



。「Emulator」窗格隨即顯示應用程式。

探索擴充的 JetChat 應用程式

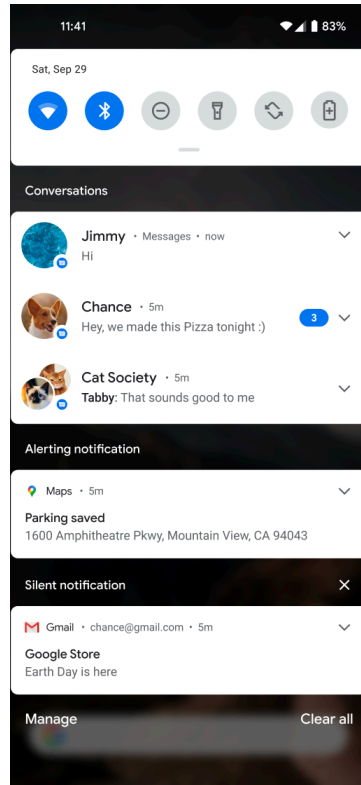
1. 在應用程式的「Message #composers」(訊息 #composers) 文字方塊中輸入訊息，然後輕觸「Send」(傳送)。
2. 離開應用程式。幾秒後，您會收到推播通知，其中包含聊天室中其他人的回覆。

注意：回覆內容並非來自真人。應用程式會模擬即時通訊體驗，並根據程式設計回覆訊息。

步驟 3 · 約 3 分鐘

3. 建立對話通知

Android 11 推出 [API](#)，可讓與即時通訊相關的通知顯示在通知導覽匣的指定專區，該專區僅用於顯示對話。



通知「必須」屬於 `Notification.MessagingStyle` 類別，並參照[長效 分享捷徑](#)。在本節中，您將瞭解如何滿足這些 API 要求，在對話專區顯示代表對話的通知。

如要建立 `NotificationCompat.MessagingStyle` 類別的通知，請按照下列步驟操作：

1. 在 Android Studio 的「Project」分頁中，依序點選「app」>「java」>「com.example.compose.jetchat」>「conversation」，然後按兩下「ConversationFragment」。
2. 在 `ConversationFragment.kt` 檔案中，找出 `ConversationFragment` 類別，然後找出 `createNotification` 函式的 `Notification` 程式碼區塊，其中會建立通知。
3. 將 `setContentText` 方法替換為 `setStyle` 方法，將通知樣式設為 `NotificationCompat.MessagingStyle` 類別。這個輔助類別會新增透過 `setContentText` 方法設定的訊息，以及與訊息相關的其他內容，例如訊息傳送時間和傳送者。

ConversationFragment.kt

```
private fun createNotification(
    notificationId: Int,
    message: Message,
    person: Person,
    shortcut: ShortcutInfoCompat,
    time: Long
): Notification? {
    ...
    .setStyle(NotificationCompat.MessagingStyle(person).addMessage(
        NotificationCompat.MessagingStyle.Message(
            message.content,
            time,
            person
        )
    )
    )
    ...
    .build()
}
```

執行應用程式

1. 執行應用程式。
 2. 在應用程式的「Message #composers」(訊息 #composers) 文字方塊中輸入訊息，然後輕觸「Send」(傳送)。
 3. 離開應用程式。您會再次收到推播通知，但樣式不同。包括訊息的顯示圖片和獨特樣式。不過，您還需要完成一些工作，通知才能在應顯示的位置顯示。
-

4. 為對話建立分享目標

您需要在通知中參照分享捷徑或[分享目標](#)。分享目標定義於 `shortcuts.xml` 檔案中，是處理以程式輔助方式定義的捷徑的進入點。你建立的捷徑代表應用程式中的對話，可讓你分享對話中的內容。

注意：您建立的分享目標會專門處理收到的訊息，並在使用者裝置上顯示為通知。您也應該為送出的訊息建立分享目標，但本程式碼研究室不會介紹這項功能。不過，適用於接收郵件分享目標的步驟和程式碼範例，也適用於外寄郵件分享目標。

定義分享目標

1. 在「Project」分頁中，對「res」目錄按一下滑鼠右鍵，然後依序選取「New」>「Directory」。
2. 在文字方塊中輸入 `xml`，然後按下 `Enter` 鍵 (macOS 為 `return` 鍵)。
3. 在 `xml` 目錄上按一下滑鼠右鍵，然後選取「File」。
4. 在文字方塊中輸入 `shortcuts.xml`，然後按下 `Enter` 鍵 (macOS 為 `return` 鍵)。
5. 在 `shortcuts.xml` 檔案中，宣告 [分享目標](#)，處理 `text/plain` 型別的資料分享作業：

`shortcuts.xml`

```
<shortcuts xmlns:android="http://schemas.android.com/apk/res/android">
  <share-target android:targetClass="com.example.compose.jetchat.NavActivity">
    <data android:mimeType="text/plain" />
    <category android:name="com.example.compose.jetchat.share.TEXT_SHARE_TARGET" />
  </share-target>
</shortcuts>
```

6. 在「Project」分頁中，依序點選 `manifests` 和 `AndroidManifest.xml`。
7. 在 `AndroidManifest.xml` 檔案中，定義 `shortcuts.xml` 檔案：

`AndroidManifest.xml`

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.compose.jetchat">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/Theme.Jetchat.NoActionBar">
...
    <meta-data android:name="android.app.shortcuts"
        android:resource="@xml/shortcuts" />

    </application>
</manifest>

```

8. 在 `AndroidManifest.xml` 檔案的 `activity` 元件中，定義包含共用邏輯的意圖篩選器：

AndroidManifest.xml

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.compose.jetchat">

...
    <activity
        ...
        <intent-filter>
            <action android:name="android.intent.action.SEND" />
            <category android:name="android.intent.category.DEFAULT" />
            <data android:mimeType="text/plain" />
        </intent-filter>
...

```

定義快速鍵

你必須為每則通知關聯相關捷徑。每個對話只能定義一個專屬捷徑，因為專屬捷徑代表要與之分享的單一聯絡人。

如要產生捷徑，請按照下列步驟操作：

1. 在「Project」分頁中，依序點選「`app`」>「`java`」>「`com.example.compose.jetchat`」>「`conversation`」>「`util`」，然後按兩下「`ConversationUtil`」。
2. 在 `ConversationUtil.kt` 檔案中，新增 `generateShortcut` 函式：

ConversationUtil.kt

```

import android.content.Intent
import androidx.core.content.pm.ShortcutInfoCompat
import com.example.compose.jetchat.NavActivity

class ConversationUtil {

    companion object {
        ...
    }
    fun generateShortcut(context: Context, shortcutId: String): ShortcutInfoCompat {
        return ShortcutInfoCompat.Builder(context, shortcutId)
            .setCategories(setOf(CATEGORY_SHARE))
            .setLongLived(true)
            .setShortLabel(shortcutId)
            .setLongLabel(shortcutId)
            .setIntent(
                Intent(context, NavActivity::class.java)
                    .setAction(Intent.ACTION_VIEW)
            )
            .build()
    }
}

```

這個函式包含對話所需的 `setPerson` 和 `setLongLived` 方法。這個人是與捷徑相關聯的聯絡人，將 `long-lived` 設為 `true` 值可確保系統快取這個捷徑，並在 UI 的各種介面中顯示。

參考通知中的快速鍵

注意：為送出的訊息發布的分享目標不會繫結至通知，因為送出的訊息通常不會觸發通知，傳送給使用者。

您需要在通知中參照分享捷徑。不過，您必須先建立捷徑，才能推送通知。

步驟如下：

1. 在 `ConversationFragment.kt` 檔案中，找出 `ConversationFragment` 類別。
2. 在呼叫 `notification` 變數之前，請建立參照 `ConversationUtil.generateShortcut` 產生的捷徑的 `shortcut` 變數。
3. 在 `notification` 變數的 `createNotification` 方法中，將 `null` 替換為 `shortcut` 變數 (做為參數)。

ConversationFragment.kt

```

private fun simulateResponseAsANotification() {
    ...
    if (message.author != "me") {
        ...
        val shortcut = ConversationUtil.generateShortcut(context!!, message.author)
        val notification = createNotification(notificationId, message, person, shortcut, time)
        ...
    }
}

```

4. 在 `createNotification` 方法中，新增 `NotificationCompat.Builder#setShortcutInfo` 方法，然後傳遞 `shortcut` 變數做為參數。

ConversationFragment.kt

```
private fun createNotification(
    notificatoinIf: Int,
    messagin: Message,
    person: Person,
    shortcut: ShortcutInfoCompat?,
    time: Long
): Notification {
    ...
    return NotificationCompat.Builder(context!!, ConversationUtil.CHANNEL_MESSAGES)
        ...
        .setShortcutInfo(shortcut)
        .build()
}
```

注意： `NotificationCompat.Builder#setShortcutInfo` 方法也會在快速鍵上設定 `LocusId` 類別，強烈建議您這麼做。接著，如果您使用 `LocusId` 類別為相關聯的活動加上註解，系統就會將信號傳送至裝置端智慧功能，指出該捷徑與該對話相關聯。系統會使用這些信號相應地排列對話，確保您根據使用者活動正確顯示對話。詳情請參閱「[最佳做法](#)」。

注意： 分享目標不限於對話。這些檔案不一定會出現在對話中。如要進一步瞭解分享目標，請參閱 [Sharing Shortcuts API](#)。

發布捷徑

- 如要發布捷徑，請在 `notificationManager.notify` 方法之前的 `simulateResponseAsNotification` 函式中，呼叫 `pushDynamicShortcut` 方法：

ConversationFragment.kt

```
import androidx.core.content.pm.ShortcutManagerCompat

...private fun simulateResponseAsNotification() {
    ...
    if (message.author != "me") {
        ...
        ShortcutManagerCompat.pushDynamicShortcut(context!!, shortcut)
        ...
    }
}
```

注意：發布捷徑的方式有很多種，但 `pushDynamicShortcut` 方法會使用 `reportShortcutUsed` API，向裝置端智慧功能發出信號，表示這是要分享內容的聯絡人。`pushDynamicShortcut` 方法也能確保捷徑更新，無須處理捷徑數量限制。

注意：如果長按 `MessagingStyle` 通知，但該通知未參照捷徑，系統會強調該通知不支援對話功能。使用參照的快速鍵，即可設定這則通知的優先順序和設定。詳情請參閱「[測試對話通知和捷徑](#)」。

執行應用程式

1. 執行應用程式。
 2. 在應用程式的「Message #composers」(訊息 #composers) 文字方塊中輸入訊息，然後輕觸「Send」(傳送)。
 3. 離開應用程式。你會再次收到推播通知，但這次的樣式更明顯，是與對話相關的通知。個人頭像圖示更顯眼，並整合了應用程式圖示。寄件者、時間和文字的顯示方式也更加簡潔。
-

5. 選用：啟用泡泡

Android 9 推出泡泡功能，並在 Android 11 中改良及重新調整用途，以使用於對話情境。泡泡是圓形疊加層，代表對話的虛擬人偶。對話泡泡會顯示在應用程式啟動器中，方便您在展開的泡泡中回覆對話。即使實作泡泡功能，是否顯示泡泡仍取決於使用者的偏好設定。

如要啟用泡泡，請按照下列步驟操作：

1. 在 `AndroidManifest.xml` 檔案中新增 `allowEmbedded` 和 `resizeableActivity` 屬性，然後將每個屬性設為 `true` 值：

AndroidManifest.xml

```
<activity
    ...
    android:allowEmbedded="true"
    android:resizeableActivity="true"
    ...
</activity>
```

2. 在 `ConversationUtil.kt` 檔案的 `ConversationUtil` 類別中，新增泡泡中繼資料：

ConversationUtil.kt

```
import androidx.core.app.NotificationCompat
import androidx.core.graphics.drawable.IconCompat

fun createBubbleMetadata(context: Context, icon: IconCompat):
NotificationCompat.BubbleMetadata {
    // Create bubble intent
    val target = Intent(context, NavActivity::class.java)
    val bubbleIntent = PendingIntent.getActivity(context, REQUEST_BUBBLE, target,
flagUpdateCurrent(mutable = true))

    // Create bubble metadata
    return NotificationCompat.BubbleMetadata.Builder(bubbleIntent, icon)
        .setDesiredHeight(400)
        .setSuppressNotification(true)
        .build()
}
```

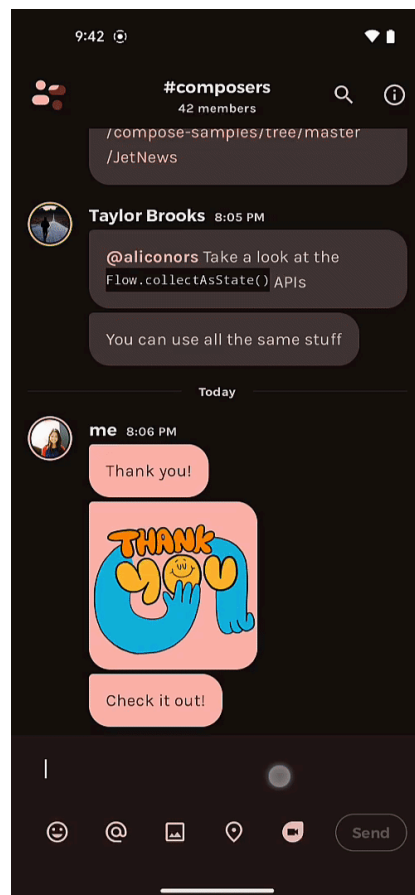
3. 在 `ConversationFragment.kt` 檔案中，建立並參照通知中的泡泡中繼資料：

ConversationFragment.kt

```
private fun createNotification(
    notificatoinIf: Int,
    messagin: Message,
    person: Person,
    shortcut: ShortcutInfoCompat?,
    time: Long
): Notification {
    ...
    // Reference the bubble metadata in the notification.
    return NotificationCompat.Builder(context!!, ConversationUtil.CHANNEL_MESSAGES)
        ...
        .setBubbleMetadata(ConversationUtil.createBubbleMetadata(context!!, person.icon!!))
    ...
    .build()
}
```

執行應用程式

1. 執行應用程式。
2. 在應用程式的「Message #composers」(訊息 #composers) 文字方塊中輸入訊息，然後輕觸「Send」(傳送)。
3. 離開應用程式。幾秒後，你會收到即時通訊通知，並以對話框形式顯示。
4. 輕觸泡泡。對話會從對話框開啟。



步驟 6 · 約 1 分鐘

6. 選用：分享連結

您已宣告分享目標，並在通知中參照這些目標，這也讓您的聯絡人顯示在 Sharesheet 中。Sharesheet 是在傳送 ACTION 意圖時顯示的由下而上元件。分享目標會顯示在分享畫面頂端，方便你在對話中分享多媒體內容。

如要叫用分享功能表，請按照下列步驟操作：

- 1. 在裝置上開啟 Google Chrome，然後前往所選網頁，例如 developer.android.com。
- 2. 如有需要，請按一下「更多直向」圖示

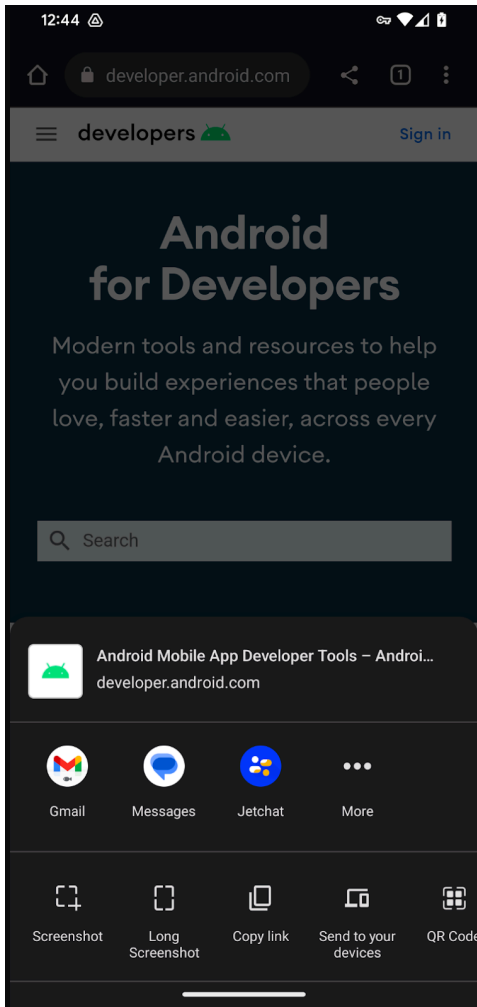


。

- 3. 點選「分享」圖示



。分享畫面會顯示在畫面底部。



- 4. 如果可以，請按一下



「JetChat」**JetChat**。系統會在即時通訊中分享網址。

- 5. 如果沒有看到



「JetChat」，請按一下



「更多」，叫出系統 Sharesheet，然後在 Sharesheet 上向上滑動，並按一下



「JetChat」。系統會在即時通訊中分享網址。

這是簡單的例子。可分享的內容類型更加豐富。詳情請參閱「[接收其他應用程式傳送的簡單資料](#)」。

步驟 7 · 約 0 分鐘

7. 恭喜

恭喜！現在您已瞭解如何使用 Messaging 和 People API，在 Android 應用程式中新增即時通訊相關功能。祝您聯絡愉快！

瞭解詳情

- [表情符號相容性](#)
 - [接收多媒體內容](#)
 - [PeopleManager](#) 類別
 - [大眾對話](#)
-